Estructuras Repetitivas y Sentencias de Ramificación en Java

Taller Práctico

Estimados estudiantes, bienvenidos al taller práctico de la Unidad 1.3. El propósito pedagógico de esta guía es afianzar el dominio de las estructuras de control repetitivas (`while` , `do-while` , `for`) y las sentencias de ramificación de flujo (`break` , `continue`) en la plataforma Java SE. A través de este conjunto de especificaciones técnicas y retos de ingeniería algorítmica, desarrollarán competencias clave para el diseño de lógica orientada a objetos de alta eficiencia, la validación estricta de entradas y el uso profesional de las herramientas de depuración (*debugger*) en IntelliJ IDEA.

1. Guía de Configuración Inicial en IntelliJ IDEA

Antes de abordar los ejercicios algorítmicos, es indispensable preparar una estructura de proyecto limpia, organizada y modular dentro del entorno de desarrollo integrado (IDE) IntelliJ IDEA:

1. **Creación del Proyecto**:
   - Abra IntelliJ IDEA y seleccione la opción **New Project**.
   - En el panel lateral, elija **Java**. Asegúrese de seleccionar una versión del SDK actualizada (Java 17 LTS o superior).
   - Asigne al proyecto el nombre exacto: `TallerEstructurasRepetitivas` .
   - Verifique que la opción para añadir código de ejemplo esté desactivada para iniciar con un proyecto totalmente limpio.
2. **Creación del Paquete Principal**:
   - En la ventana del proyecto (*Project Tool Window*), expanda la carpeta `src` .
   - Haga clic derecho sobre la carpeta `src` , seleccione **New -> Package** y nombre el paquete como `com.taller.practico` .
3. **Organización de Clases Autónomas**:
   - Para mantener una separación clara de responsabilidades, deberá crear una clase Java independiente dentro del paquete `com.taller.practico` para cada una de las secciones del taller.
   - Cada clase debe contar obligatoriamente con su propio método principal estándar: `public static void main(String[] args)` .
   - Toda interacción con el usuario a través de la consola deberá gestionarse mediante la clase `java.util.Scanner` .

2. Sección 1: Bucle `while` (Control de Flujo con Condición Previa)

El bucle `while` evalúa su condición lógica antes de ejecutar el cuerpo de instrucciones. Si la condición es falsa desde el inicio, el ciclo no se ejecutará ninguna vez.

2.1 Ejercicio 1.1: Contador y Acumulador con Valor Centinela

- **Clase asignada**: `Ejercicio1_1`
- **Especificación Técnica**: Diseñe un programa de consola que solicite continuamente números enteros al usuario.
- **Requerimientos Algorítmicos**:
  - Implemente un bucle `while` controlado por un **valor centinela**. El programa debe detener su ejecución inmediatamente cuando el usuario ingrese el número `-99`.
  - Lleve un registro exacto mediante dos variables: un **acumulador** (para la suma total de los datos válidos) y un **contador** (para la cantidad total de números válidos introducidos).
  - El valor centinela (`-99`) se utiliza exclusivamente para indicar la finalización del proceso; por lo tanto, **no debe sumarse** al acumulador ni debe ser contabilizado.
  - Al concluir el bucle, el programa debe imprimir en consola el total acumulado de los números ingresados y la cantidad exacta de entradas procesadas.

2.2 Ejercicio 1.2: Control de Proceso con Bandera de Parada (Boolean Flag)

- **Clase asignada**: `Ejercicio1_2`
- **Especificación Técnica**: Construa un sistema para la gestión de transacciones financieras consecutivas en un módulo de caja.
- **Requerimientos Algorítmicos**:
  - El control de iteración del bucle `while` debe depender del estado de una variable de control de tipo booleano (`boolean procesando = true`).
  - Solicite en cada iteración el monto de una transacción numérica.
  - La bandera booleana debe cambiar su valor a `false` interrumpiendo el ciclo si se cumple cualquiera de las siguientes condiciones de parada:
    1. El usuario ingresa un valor estrictamente negativo (monto no válido).
    2. El acumulador total acumulado de las transacciones procesadas excede el límite máximo permitido de $1'000.000 (un millón de pesos).
  - Al finalizar, el sistema debe presentar un informe por consola con el saldo final acumulado y la causa específica por la cual finalizó la captura de datos (monto negativo o superación del límite).

3. Sección 2: Bucle `do-while` (Ejecución Mínima Garantizada y Menús)

A diferencia del bucle `while`, la estructura `do-while` evalúa la condición de parada después de ejecutar el bloque de código, lo que garantiza que las instrucciones contenidas se ejecuten como mínimo una vez.

3.1 Ejercicio 2.1: Validación Estricta de Entradas del Usuario

- **Clase asignada**: `Ejercicio2_1`

- **Especificación Técnica**: Implemente un mecanismo riguroso de validación de clave de acceso de 4 dígitos para un cajero automático.
- **Requerimientos Algorítmicos**:
  - Solicite al usuario el ingreso de un PIN de seguridad (número entero positivo de 4 dígitos, en el rango entre 1000 y 9999).
  - Mediante la estructura `do-while`, garantice que el programa solicite la clave al menos una vez.
  - Si el número ingresado se encuentra fuera del rango válido de 4 dígitos, el sistema debe emitir un mensaje de alerta explicativo ("Error: La clave debe tener exactamente 4 dígitos positivos") y volver a solicitar el dato dentro de la misma iteración.
  - El ciclo finaliza únicamente cuando se suministre un valor que cumpla la condición estricta de validación.

3.2 Ejercicio 2.2: Menú Interactivo de Consola

- **Clase asignada**: `Ejercicio2_2`
- **Especificación Técnica**: Desarrolle la interfaz textual de una plataforma bancaria basada en menús interactivos.
- **Requerimientos Algorítmicos**:
  - Despliegue en consola el siguiente menú de opciones textuales:
    1. Consultar Saldo
    2. Realizar Depósito
    3. Realizar Retiro
    4. Salir del Sistema
  - Emplee una estructura `do-while` para asegurar que el menú se muestre obligatoriamente al inicio y se repita continuamente tras cada acción seleccionada.
  - Incorpore una estructura de decisión múltiple (`switch`) para simular la ejecución de la opción seleccionada.
  - El sistema solo debe permitir la terminación del ciclo cuando el usuario elija explícitamente la opción 4 ("Salir del Sistema").

4. Sección 3: Bucle `for` Tradicional (Ciclos Contados y Secuencias)

El bucle `for` tradicional es ideal para ejecutar bloques de código cuando se conoce de antemano el número exacto de iteraciones o cuando se requiere iterar dentro de un rango predefinido mediante variables de control.

4.1 Ejercicio 3.1: Generador de Tablas de Multiplicar y Conteos Regresivos

- **Clase asignada**: `Ejercicio3_1`
- **Especificación Técnica**: Diseñe un generador doble de secuencias numéricas exactas.
- **Requerimientos Algorítmicos**:

- Solicite un número entero N al usuario mediante consola.
- Emplee un primer ciclo `for` para imprimir la tabla de multiplicar de dicho número N, evaluando iterativamente del 1 al 12 con un formato estético e alineado (ej. `N x i = Resultado`).
- Posteriormente, implemente un segundo ciclo `for` totalmente independiente que genere y muestre por pantalla un conteo regresivo que inicie en el valor N introducido y finalice exactamente en 0.

4.2 Ejercicio 3.2: Cálculo Matemático y Patrones Numéricos

- **Clase asignada**: `Ejercicio3_2`
- **Especificación Técnica**: Desarrolle un algoritmo dual para computar factoriales e imprimir figuras geométricas en consola.
- **Requerimientos Algorítmicos**:
  - **Parte A**: Solicite un número entero positivo N y calcule su factorial (N!) utilizando un ciclo `for`. Tenga en cuenta el manejo de tipos de datos adecuados para evitar desbordamientos aritméticos.
  - **Parte B**: Utilizando **bucles** `**for**` **anidados** (un ciclo externo para las filas y un ciclo interno para los elementos de cada fila), imprima en la consola un patrón triangular cuya altura responda al valor N. La figura debe construirse agregando un elemento numérico o carácter especial en cada nivel (por ejemplo, fila 1 con 1 elemento, fila 2 con 2 elementos, hasta la fila N).

5. Sección 4: Sentencias de Ramificación (`break` y `continue`)

Las sentencias de ramificación modifican el flujo natural de ejecución de los bucles: `break` finaliza el ciclo de forma abrupta, mientras que `continue` omite el resto de la iteración actual y avanza a la siguiente evaluación condicional.

5.1 Ejercicio 4.1: Búsqueda e Interrupción Temprana (`break`)

- **Clase asignada**: `Ejercicio4_1`
- **Especificación Técnica**: Diseñe un algoritmo de localización de coincidencias numéricas específicas dentro de un intervalo cerrado.
- **Requerimientos Algorítmicos**:
  - Implemente un bucle que recorra un rango de números enteros del 1 al 100.
  - Evalúe mediante una condición si el número iterado es **divisible simultáneamente entre 7 y 11** (es decir, residuo igual a cero en ambas divisiones).
  - Al encontrar el **primer número** que satisfaga ambas condiciones, el programa debe imprimir dicho número por pantalla y hacer uso inmediato de la sentencia `break` para detener el bucle sin procesar los números restantes del rango.

5.2 Ejercicio 4.2: Filtrado y Salto de Registros Inválidos (`continue`)

- **Clase asignada**: `Ejercicio4_2`
- **Especificación Técnica**: Desarrolle un procesador de secuencias numéricas con reglas de filtrado exclusivas.
- **Requerimientos Algorítmicos**:
  - Configure un ciclo para iterar a través de los números enteros comprendidos entre 1 y 50.
  - Utilice la sentencia `continue` para **omitir explícitamente** la impresión y el procesamiento de:
    1. Todos los números impares.
    2. Todos los números que sean múltiplos de 5.
  - Únicamente los números pares que **no** sean múltiplos de 5 deberán ser mostrados en la consola.

6. Sección 5: Reto Integrador (Simulador de Control de Inventario)

- **Clase asignada**: `EjercicioIntegrador`
- **Especificación Técnica**: Construya un prototipo de consola para la gestión y recepción de lote de mercancías en un almacén logístico, integrando la totalidad de las estructuras repetitivas y sentencias de control aprendidas.
- **Requerimientos del Sistema Integrado**:
  1. **Navegación General (`**do-while**`)**: Encapsule toda la lógica de la aplicación en un ciclo `do-while` que despliegue un menú principal con las opciones: `1. Registrar Lote de Productos`, `2. Ver Estado del Almacén`, `3. Salir`.
  2. **Procesamiento de Lotes (`**for**`)**: Al elegir la opción de registro, el programa debe solicitar la cantidad fija de productos que componen el lote (ej. N productos) e iniciar un bucle `for` para iterar exactamente N veces.
  3. **Validación Estricta de Datos (`**while**`)**: Al ingresar el precio y la cantidad en stock de cada producto, utilice bucles `while` independientes para garantizar que ambos valores sean estrictamente mayores a cero. Si el usuario ingresa un valor negativo o cero, pida nuevamente el dato hasta que sea válido.
  4. **Filtrado de Productos Descontinuados (`**continue**`)**: Durante la iteración del lote, pregunte si el producto se encuentra en estado "descontinuado". De ser así, ejecute la sentencia `continue` para ignorar su procesamiento y no sumarlo a las estadísticas de inventario.
  5. **Parada por Capacidad Crítica (`**break**`)**: Defina un límite máximo para la capacidad total del almacén (ej. 500 unidades). Si durante el procesamiento del lote la acumulación total de unidades supera dicha capacidad, emita una alerta de sobrecupo y ejecute la sentencia `break` para abortar inmediatamente el procesamiento del lote restante.

7. Sección 6: Análisis y Depuración de Código (Debugging en IntelliJ IDEA)

A continuación, se presentan tres escenarios con errores sintácticos o lógicos típicos de programación. Ingrese cada bloque de código en clases independientes dentro de su IDE y utilice las herramientas de depuración (*Debugger*) para identificar y solucionar las fallas.

7.1 Desafío de Depuración 1: Bucle Infinito por Falta de Incremento

- **Clase asignada**: `Debug1`
- **Instrucción**: Copie el código suministrado. Al ejecutarlo, notará que el programa entra en un ciclo infinito. Utilice el botón de pausar ejecución en el IDE para inspeccionar el valor de la variable de control.

```
package com.taller.practico;

public class Debug1 {
    public static void main(String[] args) {
        int contador = 1;
        int limite = 10;

        System.out.println("Iniciando procesamiento...");

        while (contador <= limite) {
            System.out.println("Procesando elemento número: " + contador);
            // Error: Falta la actualización de la variable de control
        }

        System.out.println("Procesamiento finalizado con éxito.");
    }
}
```

7.2 Desafío de Depuración 2: Error de Desfase por Uno (*Off-by-One Error*)

- **Clase asignada**: `Debug2`
- **Instrucción**: El requerimiento dicta procesar los elementos desde el índice 1 hasta el 5 inclusive. Sin embargo, el siguiente código realiza una iteración de menos debido a un operador relacional erróneo.

```
package com.taller.practico;

public class Debug2 {
```

```
    public static void main(String[] args) {
        int totalElementos = 5;

        System.out.println("Imprimiendo elementos del 1 al " + totalElementos +
":");

        // Error: Condición de parada con operador relacional incorrecto (< en
lugar de <=)
        for (int i = 1; i < totalElementos; i++) {
            System.out.println("Elemento " + i);
        }
    }
}
```

7.3 Desafío de Depuración 3: Uso Indebido o Desubicado de `break`

- **Clase asignada**: `Debug3`
- **Instrucción**: El siguiente algoritmo tiene como objetivo procesar únicamente números pares. Sin embargo, la ubicación incorrecta de la sentencia `break` provoca que el ciclo aborte prematuramente en la primera iteración.

```
package com.taller.practico;

public class Debug3 {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i % 2 != 0) {
                System.out.println("Número impar detectado: " + i);
                break; // Error: Se utilizó break en lugar de continue dentro del
condicional
            }
            System.out.println("Número par procesado: " + i);
        }
    }
}
```

7.4 Instrucciones de Ejecución de Depuración en IntelliJ IDEA

Para llevar a cabo el análisis visual de las variables durante la ejecución en tiempo de real, siga minuciosamente estos pasos en su entorno IntelliJ IDEA:

1. **Establecer Puntos de Interrupción (*Breakpoints*)**:

- Abra la clase que desea depurar (por ejemplo, `Debug1.java`).
- Haga clic con el botón izquierdo del ratón en el **margen izquierdo** (al lado del número de línea), en la instrucción donde inicia el bucle. Aparecerá un círculo rojo indicando que el *breakpoint* está activo.

2. **Iniciar la Ejecución en Modo Depuración**:
   - En lugar de ejecutar el programa de forma estándar, use el atajo de teclado `**Shift + F9**` (o haga clic en el icono con forma de insecto / *bug* en la esquina superior derecha).
   - La ejecución se pausará automáticamente en la línea donde ubicó el *breakpoint*, resaltándola en color amarillo.
3. **Inspección de Variables**:
   - Abra la pestaña **Variables** en la ventana del depurador (*Debug Window*) situada en la parte inferior de la pantalla.
   - Observe los valores actuales de las variables locales (`contador`, `limite`, `i`) en tiempo real.
4. **Avance Paso a Paso**:
   - Presione la tecla `**F8**` (*Step Over*) para ejecutar la línea de código actual y avanzar a la siguiente instrucción sin entrar en métodos externos.
   - Evalúe cómo cambian los valores en la pestaña *Variables* tras presionar `F8` repetidamente. Esto le permitirá vislumbrar exactamente por qué se producen el bucle infinito, el error de desfase o la parada prematura, dándole la pauta para corregir la sintaxis requerida.